



**PROJETO**







- LLMs, ou Grandes Modelos de Linguagem, são sistemas de inteligência artificial avançados, treinados em grandes quantidades de dados de texto para compreender e gerar linguagem humana



- LangChain é um framework de código aberto para desenvolver aplicações que utilizam modelos de linguagem grandes (LLMs).



LangChain

# LLM APPLICATION LANGUAGE TRANSLATION

CRIAÇÃO DE UM TRADUTOR PYTHON + IA(LLM = LANGCHAIN)

# LangServe Playground

Try it

Inputs

Reset

IDIOMA\*

french

TEXTO\*

bom dia

Output

Bonjour

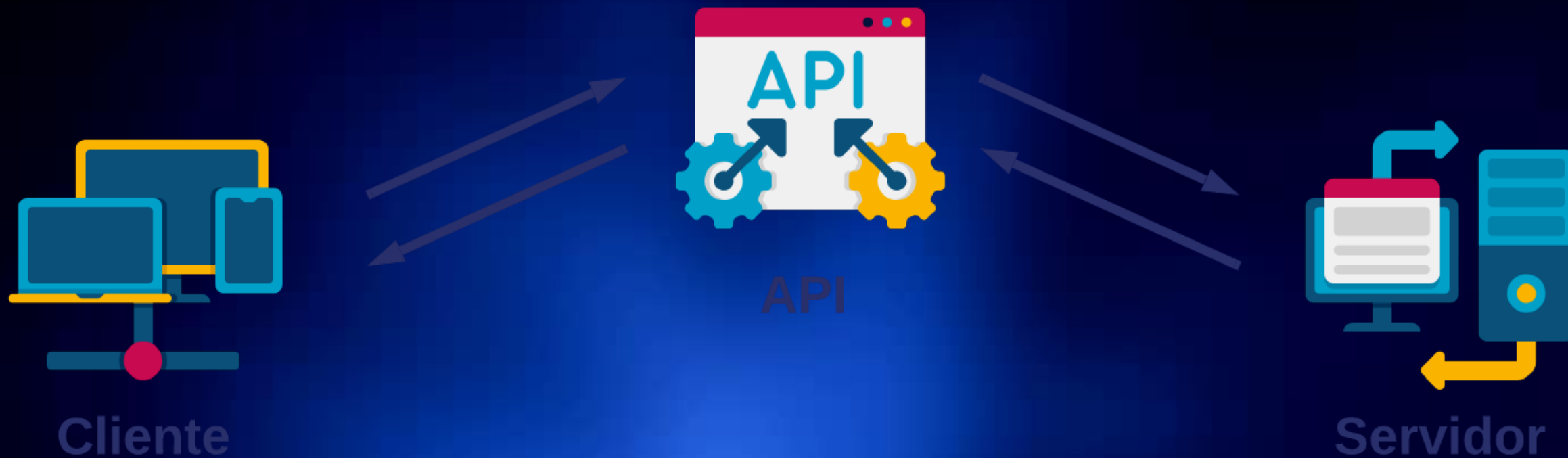
Intermediate steps 3



 Share

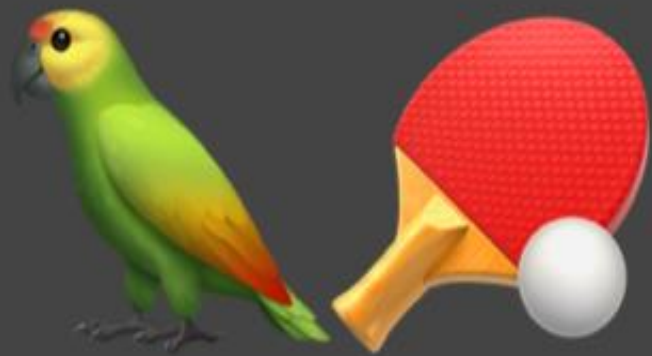
 Start

# Visão geral do projeto



Cofinanciado por:





# LangServe



LLM

Códigos

```
1  # Importa o objeto 'chain' do módulo 'codigo'.
2  # Esse objeto provavelmente representa uma cadeia de processamento de linguagem (como uma pipeline de tradução usando LangChain).
3  from codigo import chain
4
5  # Importa a classe FastAPI, que é usada para criar a aplicação web.
6  from fastapi import FastAPI
7
8  # Importa a função 'add_routes' do pacote 'langserve', que permite expor chains do LangChain como rotas de API.
9  from langserve import add_routes
10
11 # Cria uma instância da aplicação FastAPI, com um título e descrição personalizados.
12 app = FastAPI(
13     title="Meu app de IA",
14     description="Traduza o texto que você quiser para qualquer língua"
15 )
16
17 # Adiciona rotas à aplicação FastAPI usando o objeto 'chain'.
18 # O caminho (endpoint) será '/tradutor', e o LangServe irá lidar com as requisições a esse endpoint.
19 add_routes(app, chain, path="/tradutor")
20
21 # Este bloco só será executado se o arquivo for executado diretamente (não importado como módulo).
22 if __name__ == "__main__":
23     # Importa o módulo uvicorn, que é um servidor ASGI usado para rodar aplicações FastAPI.
24     import uvicorn
25
26     # Inicia o servidor uvicorn, fazendo a aplicação estar disponível em http://localhost:8000
27     uvicorn.run(app, host="localhost", port=8000)
28
```

 código.py > ...

```
1  # disponibilizar video de variavel de ambiente (dotenv)
2  # disponibilizar video de FastAPI
3
4  from dotenv import load_dotenv
5  import os
6  from langchain_core.messages import SystemMessage, HumanMessage
7  from langchain_core.output_parsers import StrOutputParser
8  from langchain_openai import ChatOpenAI
9  from langchain_core.prompts import ChatPromptTemplate
10
11  load_dotenv()
12  chave_api = os.getenv("OPENAI_API_KEY")
13
14  mensagens = [
15      SystemMessage("Traduza o texto a seguir para inglês"),
16      HumanMessage("Se inscrevam no canal para aprender Python")
17  ]
18
19  modelo = ChatOpenAI(model="gpt-4o")
20  parser = StrOutputParser()
21  chain = modelo | parser
22
23  # resposta = modelo.invoke(mensagens)
24
25  # texto = parser.invoke(resposta)
26
27  template_mensagem = ChatPromptTemplate.from_messages([
28      ("system", "Traduza o texto a seguir para {idioma}"),
29      ("user", "{texto}"),
30  ])
31
32  # print(template_mensagem.invoke({"idioma": "francês", "texto": "Dê like no vídeo e comente o que você tá achando"}))
33
34  chain = template_mensagem | modelo | parser
35  # texto = chain.invoke({"idioma": "francês", "texto": "Dê like no vídeo e comente o que você tá achando"})
36
37  # print(texto)
38
```

 cliente.py > ...

```
1  # Importa a classe RemoteRunnable da biblioteca langserve, usada para se conectar a uma chain remota.
2  from langserve import RemoteRunnable
3
4  # Cria uma instância de RemoteRunnable apontando para o endpoint de uma API LangChain que está rodando localmente na rota /tradutor.
5  chain_remota = RemoteRunnable("http://localhost:8000/tradutor")
6
7  texto = chain_remota.invoke({ # Chama (invoca) a chain remota enviando um dicionário com os dados necessários para a tradução.
8      "idioma": "espanhol",      # Especifica que o idioma de destino da tradução é o espanhol.
9      "texto": "Já deu like no vídeo? Se não, dá agora" # Texto em português que será traduzido.
10 })
11
12 print(texto) # Exibe no terminal o texto traduzido retornado pela chain remota.
```